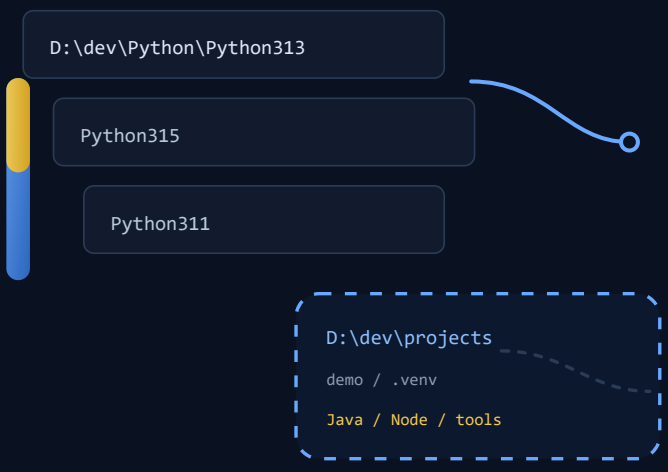
D:\DEV\PYTHON / MULTI-VERSION

py.

# 多版本 Python 与开发目录

把每个 Python 版本装进 `D:\dev\Python` 的独立子目录，让解释器、项目、Java 与其他工具共用一套清晰的开发盘结构。修订版重点更新安装路径、目录规划与迁移步骤。

WINDOWS 10 / 11 · macOS 12+ · LINUX



READ ME FIRST / 使用说明

# 关于本指南

Python 默认装到 `C:\Program Files` 或用户目录，适合“只装一个版本”的机器。但开发电脑往往还要装 JDK、Maven、Node、数据库客户端，C 盘和 Program Files 会越来越难管理。本指南统一采用 `D:\dev` 开发目录，把每个 Python 版本放进独立子目录。

| DOCUMENT | BASE DIR | UPDATED |
| --- | --- | --- |
| **多版本 Python 与开发目录** | `D:\dev\Python`<br>每个版本独立子目录 | **2026 年 10 月**<br>修订版 v1.1 |

## 适合谁读

- 新电脑准备把 Python 与 Java、Node 等统一放在开发盘的人。
- 已经有多版本 Python，但路径散落在 C 盘各处的人。
- 希望项目、解释器、工具链互不干扰的开发者。
- 需要给团队写新机初始化规范的人。

## 本指南覆盖什么

- D:\dev 开发目录规划，以及 D:\dev\Python\Python3xx 的安装路径。
- Windows 安装器逐项设置、py 启动器、PATH 与多版本共存。
- macOS / Linux 的对应路径与迁移思路。
- venv、pip、IDE、验收与故障排查。

## 怎么读最省时间

ROUTE 01
**全新电脑，第一次装**
先读第 1 章建目录，再按第 3 章安装到 D:\dev\Python，最后做第 9 章验收。

ROUTE 02
**已有版本，但路径很乱**
看 1.4 节整理旧版本，再用 3.6 节确认安装目录与 py 注册。

ROUTE 03
**只关心虚拟环境**
从第 5 章开始；第 6–7 章讲 pip、项目结构和 IDE。

ROUTE 04
**还要配 Java / Node**
先看 1.2 节目录树，再看 8.3 节的 JDK、Maven、Node 预留方案。

01 **解释器各归其位**
D:\dev\Python\Python3xx

02 **项目环境隔离**
D:\dev\projects\name\.venv

03 **工具链可扩展**
Java / Node / tools 各自分层

本版路径约定：Python 安装到 `D:\dev\Python\Python3xx`，项目放在 `D:\dev\projects`；Java、Node 等后续环境使用 `D:\dev\Java`、`D:\dev\Node`。

# 目录

CONTENTS / REVISED EDITION

PART I 目录与安装

01 先建 D:\dev：开发目录规划 5
1.1 为什么不再装进 Program Files 5
1.2 推荐的目录树 5
1.3 创建、权限与盘符选择 6
1.4 现有散落版本怎么整理 7

02 版本与工具决策 8
2.1 2026 年该装哪个版本 8
2.2 支持周期与 EOL 8
2.3 py、pyenv、uv、conda 怎么选 9

03 Windows：安装到 D:\dev\Python 10
3.1 创建目录与下载安装器 10
3.2 安装器逐项设置（重点路径） 10
3.3 py 启动器与多版本 11
3.4 PATH 与 Microsoft Store 别名 12
3.5 安装第二个版本与升级 12

04 macOS / Linux 的对应做法 14
4.1 macOS：Homebrew、pkg、pyenv 14
4.2 Linux：系统包与 pyenv 15
4.3 跨平台目录原则 15

PART II 环境与项目

05 venv：项目级隔离 16
5.1 venv 做了什么 16
5.2 创建、激活与退出 17
5.3 绑定解释器与重建 17

06 pip、依赖与可复现 19
6.1 永远用 python -m pip 19
6.2 requirements 与 pyproject 19
6.3 镜像、代理与证书 20

07 项目工作流与 IDE 21
7.1 一个项目一个 .venv 21
7.2 目录结构与 .gitignore 21
7.3 VS Code、PyCharm、Jupyter 22

08 多版本并用与工具链 23
8.1 测试多个版本 23
8.2 uv 与 conda 23

8.3 同时配置 Java / Node 的目录 24

PART III 验收与维护

09 验收、诊断与排错 25

9.1 十二项验收 25

9.2 诊断命令 26

9.3 十五个高频报错 26

10 维护、迁移与清理 28

10.1 版本升级与回滚 28

10.2 换电脑迁移清单 28

10.3 清理与磁盘管理 28

APPENDICES

A 命令速查 30

B 初始化脚本 31

C 版本、资源与术语 33

## 按问题找章节

| 你遇到的问题 | 先看 | 再查 |
| --- | --- | --- |
| 安装器默认装到 C 盘，想改到 D:\dev | 3.2 | 3.6 |
| `py -0p` 看不到新版本 | 3.3 | 9.3 |
| 创建 venv 失败 / 激活被禁止 | 5.2 | 9.3 |
| `pip` 装错环境 / 包找不到 | 6.1 | 9.3 |
| VS Code / Jupyter 解释器不对 | 7.3 | 9.3 |
| 同时配置 Java / Node | 8.3 | 10.4 |
| 换电脑重建环境 | 10.2 | 附录 B |

按“症状 → 机制 → 排错”顺序阅读，比从头翻更省时间。

页码为条目所在页；点击条目可跳转。目录在排版完成后自动生成。

LAYOUT / 先建目录

01

# 先建 D:\dev：开发目录规划

安装 Python 之前，先把“开发工具放哪里”定下来。目录结构一旦被 JDK、Maven、Node、Python 和项目共用，就不应该按某个软件的默认值随手决定。

预计 15 分钟 | Windows 主路径 | 可扩展到 Java / Node

## 1.1 为什么不再装进 Program Files

默认安装路径不是不能用，而是不适合“多版本 + 多工具链”的开发机。把 Python、JDK、Node 都塞进 `C:\Program Files`，会带来四个长期问题：

- **权限：**Program Files 默认需要管理员写入；IDE、构建工具、pip 缓存与插件经常需要修改安装目录附近的内容。
- **路径长度：**工具链层层嵌套（JDK、Maven、Gradle、Python、venv）后，路径会迅速变长，旧工具更容易触发 260 字符限制。
- **多版本：**同一个软件装多个版本时，Program Files 下的目录名容易重复、覆盖，升级时难以保留旧版本。
- **备份与迁移：**系统盘重装时，开发环境与项目分散在 C 盘各处，无法整体打包带走。

> **开发盘不是“另一个 Program Files”**
>
> `D:\dev` 是你自己拥有的目录：结构可读、可以整体备份、可以按工具分目录，也不需要每次安装都提权。它不是系统目录，卸载/重装时更容易清理。

## 1.2 推荐的目录树

核心原则：**一个工具一个顶层目录，一个版本一个子目录。**Python 版本放在 `D:\dev\Python\Python3xx`，不要把多个版本直接混在 `D:\dev\Python` 根下。

```
D:\dev\
├── Python\
│   ├── Python311\          # 旧项目兼容
│   ├── Python313\          # 主力版本
│   └── Python315\          # 尝鲜 / 新项目
├── Java\# 以后放 JDK、Maven、Gradle
│   ├── jdk-21\
│   └── jdk-17\
├── Node\# 以后放 Node、pnpm
├── tools\# Git、CMake、数据库客户端等
└── projects\
    ├── demo\
    │   ├── .venv\        # 项目自己的环境
    │   └── ...
    └── another-project\
```

| 目录 | 放什么 | 不要放什么 |
| --- | --- | --- |
| `D:\dev\Python\Python313` | 一个完整 Python 安装 | 项目代码、多个版本、venv |
| `D:\dev\projects\demo` | 项目源码与 `.venv` | Python 安装目录、全局包 |
| `D:\dev\Java\jdk-21` | 一个 JDK 版本 | Maven 仓库、项目代码 |
| `D:\dev\tools` | Git、CMake、数据库客户端 | Python 解释器、项目依赖 |

**表 1-1**　开发目录职责。工具与项目分开，备份与清理都更简单。

## 1.3 创建、权限与盘符选择

先确认 D 盘存在、剩余空间足够（Python 多版本 + 依赖建议预留 20–30 GB），再创建目录。用 PowerShell：

```
PS> New-Item -ItemType Directory -Force D:\dev\Python, D:\dev\projects, D:\dev\Java, D:\dev\Node,
D:\dev\tools | Out-Null
PS> Get-ChildItem D:\dev

# 检查当前账户是否有写权限
PS> New-Item -ItemType File D:\dev\.write-test -Force | Out-Null
PS> Remove-Item D:\dev\.write-test
```

如果创建失败，说明当前账户对 D 盘根目录没有写权限。用管理员 PowerShell 创建 `D:\dev`，再给当前用户“修改”权限：

```
PS> icacls D:\dev /grant "$env:USERNAME:(OI)(CI)M" /T
```

不要给 `Everyone` 完全控制，也不要把开发目录设在需要频繁提权的路径。个人电脑建议 D 盘为本地 NTFS；不要放在 OneDrive、百度网盘同步目录或网络映射盘。

DISK

**盘符与空间**

D 盘至少留 20 GB；如果还要装 Android SDK、Docker 镜像，建议 50 GB 以上。开发目录不要与系统盘共用剩余空间。

PATH

**路径尽量短**

`D:\dev\Python\Python313` 比深层用户目录短；不要在 `D:\dev` 下再套三层中文目录。

## 1.4 现有散落版本怎么整理

如果机器上已经有类似 `D:\dev\Python\Python312`、`D:\dev\Python\python.exe` 或 `C:\Users\...\Python311` 的版本，不要直接剪切目录。安装过的 Python 在注册表、py 启动器、开始菜单和 pip 脚本里都有记录，移动后会留下失效路径。

| 现状 | 建议 | 原因 |
| --- | --- | --- |
| 已经在 `D:\dev\Python\Python312` | 保留，登记到 py | 路径已符合规范，不需要重装。 |
| 只有一个 `D:\dev\Python\python.exe` | 确认版本后移到独立子目录或重装 | 根目录混放会让后续版本无法区分。 |
| 装在 `C:\Users\...\Programs\Python` | 卸载后用新路径重装 | 避免注册表与 PATH 留下旧路径。 |
| 便携版 / 嵌入式版 | 单独放 `D:\dev\Python\portable-3.13` | 不注册 py，需手动管理 PATH。 |

**表 1-2**　现有版本整理。移动安装目录通常比重新安装更麻烦。

> **不要为了“整齐”直接重命名目录**
>
> 重命名 `Python313` 后，`py -0p`、开始菜单、pip 的 `Scripts` 入口与 IDE 解释器都可能失效。要么用官方安装器重装到新路径，要么保留原名并记录到文档。

VERSIONS / 安装之前

02

# 版本与工具决策

目录定了，接下来决定装哪几个版本、用什么工具管理。版本不是越多越好；新电脑保留 2–3 个有明确用途的版本即可。

预计 12 分钟 | 2026-10 | 版本与 EOL

## 2.1 2026 年该装哪个版本

| 版本 | 状态 | 建议 | 说明 |
| --- | --- | --- | --- |
| **3.15** | 最新稳定 | 尝鲜 / 新项目 | 新特性最多；部分二进制包可能尚未适配，生产先验证。 |
| **3.14** | bugfix | 新项目候选 | 发布满一年，生态较稳；支持到 2030-10。 |
| **3.13** | security | **主力基线** | 轮子覆盖广，支持到 2029-10。 |
| **3.12** | security | 旧项目兼容 | 支持到 2028-10。 |
| **3.11** | security | 维护旧项目 | 支持到 2027-10。 |
| **3.10** | **EOL** | 不建议 | 2026-10-01 已结束支持。 |

**表 2-1** 新电脑建议装 3.13 + 3.15；数据科学项目按依赖选择 3.12/3.13。

最新稳定版是 **3.15.2**（2026-10-25），但“最新”不等于“所有包都能装”。PyTorch、NumPy、数据库驱动等二进制包通常滞后于新版本；生产环境优先选团队已验证的 3.13/3.14。

## 2.2 支持周期与 EOL

```
3.15  2026-10 发布   bugfix 至 2028-10   security 至 2031-10
3.14  2025-10 发布   bugfix 至 2027-10   security 至 2030-10
3.13  2024-10 发布   security 至 2029-10
3.12  2023-10 发布   security 至 2028-10
3.11  2022-10 发布   security 至 2027-10
3.10  2021-10 发布   2026-10-01 EOL
```

EOL 之后不再有安全补丁。新项目不要从 3.10 开始；旧项目至少制定迁移计划。用 `python -m pip index versions 包名` 与 `--dry-run` 检查依赖是否支持目标版本。

## 2.3 py、pyenv、uv、conda 怎么选

| 工具 | 职责 | 适合 |
| --- | --- | --- |
| **py** | Windows 启动器，按版本号调用解释器 | Windows 多版本共存；不负责下载/建环境。 |
| **pyenv** | 下载/编译多个解释器，按目录切换 | macOS/Linux 需要 3.10–3.15 频繁切换。 |
| **uv** | 解释器、环境、依赖、锁文件 | 新项目、CI、追求速度。 |
| **conda** | 解释器 + 非 Python 二进制依赖 | 数据科学、CUDA、GDAL。 |
| **venv** | 标准库项目级隔离 | 所有项目的基础做法。 |

**表 2-2** 选一个解释器管理器，再给每个项目建环境；不要把所有工具都叠在同一项目上。

> **本指南的默认组合**
>
> Windows：官方安装器 + `py` + 每项目 `venv`。macOS：Homebrew 或 python.org + venv。Linux：系统包 + venv；需要新版本用 pyenv/deadsnakes。数据科学另用 conda。

WINDOWS / CUSTOM DESTINATION

03

# Windows：安装到 D:\dev\Python

这一章只做一件事：把官方安装器从“默认路径”改成 `D:\dev\Python\Python313`，并确保第二个版本装进 `Python315`，而不是覆盖前一个。路径选对，后面的 py、PATH、venv 都会顺很多。

预计 30 分钟　需要写 D 盘权限　Windows 10 / 11

## 3.1 创建目录与下载安装器

先建目录，再下载。顺序反过来也可以，但在安装器里“浏览”到一个不存在的深层目录容易多点错。

```
PS> New-Item -ItemType Directory -Force `
    D:\dev\Python\Python313, `
    D:\dev\Python\Python315, `
    D:\dev\projects | Out-Null

PS> Get-ChildItem D:\dev\Python
```

从 `python.org/downloads/windows/` 下载目标版本的 **Windows installer (64-bit)**。ARM 设备选 ARM64。下载后右键查看数字签名，必要时用 `Get-FileHash` 核对 SHA-256。

> **目录名不要重复**
>
> 如果你已经建了 `D:\dev\Python\Python313`，安装器里目标路径就填这一层，不要再让它自动追加一个 `Python313`。最终应存在 `D:\dev\Python\Python313\python.exe`，而不是 `...\Python313\Python313\python.exe`。

## 3.2 安装器逐项设置（重点路径）

1. **第一屏：launcher 勾选，PATH 不勾**

   勾选 **Install launcher for all users (recommended)**；**Add python.exe to PATH** 不勾。多版本机器让 `py` 选版本，而不是把每个 `python.exe` 塞进 PATH。

2. **Customize installation**

   进入 Optional Features，保留 **pip** 与 **py launcher**；不需要 IDLE 可不勾 tcl/tk。点 Next 进入 Advanced Options。

3. **Advanced Options：改安装路径**

   取消 **Add Python to environment variables**；在目标路径处点 Browse，选择 `D:\dev\Python\Python313`。如果勾选 “Install for all users”，路径仍可自定义，但需要管理员权限。

**4** **确认后安装**

安装器摘要页应显示 `D:\dev\Python\Python313`。安装完成后不要急着勾“Disable path length limit”之外的选项；先验证路径。

| 安装器选项 | 设置 | 理由 |
|---|---|---|
| Install launcher for all users | **勾选** | 全局 `py`，所有账户可用。 |
| Add python.exe to PATH | **不勾** | 避免多个版本争抢 `python`；用 `py -3.x`。 |
| Customize installation → pip | **勾选** | 包管理器。 |
| py launcher | **勾选** | 多版本入口。 |
| Associate files with Python | 不勾 | 避免双击 .py 时版本不一致。 |
| Add Python to environment variables | **不勾** | 与第一屏 PATH 选项重复，统一不勾。 |
| Precompile standard library | 勾选 | 首次启动更快。 |
| Install for all users | 按需 | 个人电脑可不勾；公司统一环境可勾。 |

**表 3-1** 安装器设置。最关键的是目标路径与“不把每个版本都加 PATH”。

## 3.3 py 启动器与多版本

安装完成后打开新的 PowerShell，运行：

```
PS> py -0p
 -V:3.13 *        D:\dev\Python\Python313\python.exe

PS> py -3.13 --version
PS> py -3.13 -c "import sys; print(sys.executable)"
D:\dev\Python\Python313\python.exe
```

如果路径不是 `D:\dev\Python\Python313`，说明安装时路径没改成功，或者你看到的是旧版本。先不要继续装第二个版本，回到“应用和功能”卸载错误路径的版本，再重装。

设置默认版本：

```
# 仅当前终端
PS> $env:PY_PYTHON = "3.13"

# 写入用户环境变量，重开终端生效
PS> [Environment]::SetEnvironmentVariable("PY_PYTHON", "3.13", "User")

# 或 %LOCALAPPDATA%\py.ini
[defaults]
python=3.13
```

> **py 不读取 pyenv 的 .python-version**
>
> `.python-version` 是 pyenv/uv 的约定，Windows 官方 `py` 启动器不保证读取。要用项目固定版本，写进脚本：`py -3.13 -m venv .venv`，或使用 uv/pyenv-win。

## 3.4 PATH 与 Microsoft Store 别名

不要把 `D:\dev\Python\Python313` 和 `...\Scripts` 直接加到 PATH，除非某个工具强制要求。正确做法是用 `py`：

```
PS> py -3.13 -m venv D:\dev\projects\demo\.venv
PS> py -3.15 -m venv D:\dev\projects\demo315\.venv
```

如果 IDE 或旧脚本只认 `python`，只给**一个主版本**加入 PATH（例如 3.13 的根目录与 Scripts），不要把所有版本都加。修改后重开终端，运行 `where.exe python` 检查第一条。

若输入 `python` 弹出 Microsoft Store，关闭应用执行别名：设置 → 应用 → 高级应用设置 → 应用执行别名 → 关闭 `python.exe` 与 `python3.exe`。这比在 PATH 里和商店别名“抢顺序”更干净。

| 检查 | 期望 | 异常处理 |
| --- | --- | --- |
| `py -0p` | 显示 `D:\dev\Python\Python313` | 路径错误：卸载重装，别重命名目录。 |
| `where.exe python` | 没有输出，或只有主版本 | 多条：清 PATH，保留一个。 |
| `python -c "import sys; print(sys.executable)"` | 指向你期望的入口 | 弹出商店：关闭别名。 |

**表 3-2** Windows 入口检查。先看路径，再看版本。

## 3.5 安装第二个版本与升级

重复 3.1–3.2，把目标路径改为 `D:\dev\Python\Python315`。装完再次运行 `py -0p`，应看到两个版本：

```
PS> py -0p
 -V:3.15 *        D:\dev\Python\Python315\python.exe
 -V:3.13          D:\dev\Python\Python313\python.exe
```

带 `*` 的是默认版本，通常是最后安装的。用 `PY_PYTHON` 或 `py.ini` 把默认固定为 3.13；需要尝鲜时显式 `py -3.15`。

UPGRADE

**补丁升级**

3.13.7 → 3.13.8：下载同一次版本安装器，覆盖安装到原路径。不要先卸载，否则 site-packages 与脚本可能丢失。

UPGRADE

**次版本升级**

3.13 → 3.15：装到新的 `Python315` 目录，不要覆盖。旧项目继续用 3.13，新项目再迁。

卸载某一个版本：设置 → 应用 → 已安装的应用 → Python 3.x → 卸载。卸载器会移除该路径与注册信息，但不会删除 `D:\dev\projects` 里的项目或 `.venv`；这些环境会失效，需要重建。

**不要直接删 D:\dev\Python\Python313**

直接删除目录会留下 py 启动器记录、开始菜单快捷方式和 PATH 条目，`py -0p` 可能显示一个不存在的路径。始终用安装器的卸载入口。

## 3.6 安装目录验收

安装完成后，先验收目录，再装第二个版本。下面五项能避免“看起来装好了，实际路径不对”。

| 检查 | 命令 / 位置 | 期望 |
| --- | --- | --- |
| 安装目录 | 资源管理器打开 `D:\dev\Python\Python313` | 能看到 `python.exe`、`Lib`、`Scripts` |
| py 注册 | `py -0p` | 路径为 `D:\dev\Python\Python313\python.exe` |
| 版本 | `py -3.13 --version` | 3.13.x |
| pip | `py -3.13 -m pip --version` | 路径位于 Python313 下 |
| 未重复嵌套 | 检查路径 | 不是 `...\Python313\Python313\python.exe` |

**表 3-3** 目录验收。路径不对时先卸载重装，不要继续装 3.15。

MACOS / LINUX

04

# macOS / Linux 的对应做法

macOS 与 Linux 的包管理器不鼓励把解释器塞进任意自定义目录。对应做法是：工具链由 Homebrew/pyenv/系统包管理，项目与数据统一放 `~/dev`，保持与 Windows 的 `D:\dev` 同构。

预计 18 分钟 | Homebrew / pyenv | 不要动系统 Python

## 4.1 macOS：Homebrew、pkg、pyenv

先看系统 Python，不要替换 `/usr/bin/python3`：

```
$ python3 --version
$ which -a python3
$ uname -m
```

推荐把项目与工具放在 `~/dev`，与 Windows 的 `D:\dev` 对应：

```
~/dev/
├── projects/
│   └── demo/
├── Python/              # 可选：pyenv root 或脚本
├── Java/
└── tools/
```

安装主版本（Homebrew）：

```
$ brew install python@3.13
$ which -a python3.13
$ /opt/homebrew/bin/python3.13 --version
```

需要多版本时用 pyenv，并可选把根目录放进 `~/dev/Python/pyenv`：

```
$ export PYENV_ROOT="$HOME/dev/Python/pyenv"
$ export PATH="$PYENV_ROOT/bin:$PATH"
$ eval "$(pyenv init -)"

$ pyenv install 3.13.7
$ pyenv install 3.15.2
$ pyenv global 3.13.7
$ pyenv local 3.15.2
```

把上述三行写入 `~/.zshrc`，再 `exec zsh`。pyenv 的版本实际在 `$PYENV_ROOT/versions/` 下，仍然是一个版本一个目录，符合本指南原则。

## 4.2 Linux：系统包与 pyenv

系统 Python 不要动；先装 `python3-venv`，项目放 `~/dev/projects`：

```
$ sudo apt install python3 python3-venv python3-pip
$ mkdir -p ~/dev/projects ~/dev/tools
$ python3 -m venv ~/dev/projects/demo/.venv
```

需要 3.13/3.15 时，Ubuntu 用 deadsnakes，或 pyenv 编译安装。pyenv 根目录同样可以指到 `~/dev/Python/pyenv`：

```
$ export PYENV_ROOT="$HOME/dev/Python/pyenv"
$ export PATH="$PYENV_ROOT/bin:$PATH"
$ eval "$(pyenv init -)"
$ pyenv install 3.13.7
$ pyenv global 3.13.7
```

> **Linux 不要用自定义前缀装系统级 Python**
>
> 把 Python 装到 `/opt/dev/Python` 再改 `/usr/bin/python3` 的链接，会影响 apt、桌面与系统脚本。需要新版本时用版本化命令、pyenv 或容器，不要动系统解释器。

## 4.3 跨平台目录原则

| 平台 | 开发根目录 | 解释器管理 |
| --- | --- | --- |
| Windows | `D:\dev` | 官方安装器装到 `D:\dev\Python\Python3xx` |
| macOS | `~/dev` | Homebrew / python.org / pyenv |
| Linux | `~/dev` | 系统包 / pyenv / deadsnakes |

**表 4-1** 不同平台的“开发根目录”不同，但“工具/版本/项目分层”的原则一致。

无论哪个平台，项目路径都建议使用 `dev/projects/项目名`，并在项目内建 `.venv`。这样换机器时，只需重建环境，不需要记住每个项目散落在哪里。

VENV / PROJECT ENV

05

# venv：项目级隔离

解释器按版本放在 `D:\dev\Python`，依赖按项目放在 `D:\dev\projects\项目\.venv`。这两层分开，才能做到“升级 Python 不影响别的项目，装包不污染全局”。

预计 20 分钟 | 跨平台 | 每个项目一个环境

## 5.1 venv 做了什么

`python -m venv .venv` 会创建一个包含 `pyvenv.cfg`、解释器入口和独立 `site-packages` 的目录。它不复制标准库，也不下载新 Python；版本由创建它的基础解释器决定。

```
D:\dev\projects\demo\
├── .venv\
│   ├── pyvenv.cfg        # 基础解释器路径、版本
│   ├── Scripts\
│   │   ├── python.exe
│   │   └── Activate.ps1
│   └── Lib\site-packages\
├── src\
└── pyproject.toml
```

验证环境是否生效：

```
(.venv) PS> python -c "import sys; print(sys.executable); print(sys.prefix); print(sys.base_prefix)"
D:\dev\projects\demo\.venv\Scripts\python.exe
D:\dev\projects\demo\.venv
D:\dev\Python\Python313
```

`sys.prefix` 是环境，`sys.base_prefix` 是基础解释器。两者不同，说明 venv 生效；两者相同，说明你还在全局 Python。

## 5.2 创建、激活与退出

```
# 进入项目，指定版本创建环境
PS> cd D:\dev\projects\demo
PS> py -3.13 -m venv --upgrade-deps .venv

# 激活
PS> .\.venv\Scripts\Activate.ps1
(.venv) PS> python --version
(.venv) PS> python -m pip --version

# 退出
(.venv) PS> deactivate
```

| Shell | 激活 | 说明 |
| --- | --- | --- |
| PowerShell | `.\.venv\Scripts\Activate.ps1` | 可能需 `RemoteSigned` 执行策略。 |
| CMD | `.\.venv\Scripts\activate.bat` | 不要用 ps1。 |
| Git Bash | `source .venv/Scripts/activate` | 路径用斜杠。 |
| macOS/Linux | `source .venv/bin/activate` | 提示符出现 `(.venv)`。 |

**表 5-1** 激活方式。激活只是修改当前 shell 的 PATH，不是“启动一个 Python”。

不激活也能用绝对路径，适合脚本与 CI：

```
PS> D:\dev\projects\demo\.venv\Scripts\python.exe -m pip install -r requirements.txt
PS> D:\dev\projects\demo\.venv\Scripts\python.exe app.py
```

## 5.3 绑定解释器与重建

venv 创建后不能通过激活切换版本。要把项目从 3.13 升到 3.15，必须重建：

```
PS> cd D:\dev\projects\demo
PS> deactivate
PS> Remove-Item -Recurse -Force .venv
PS> py -3.15 -m venv --upgrade-deps .venv
PS> .\.venv\Scripts\Activate.ps1
(.venv) PS> python -m pip install -r requirements.txt
```

重建前先导出/提交依赖。虚拟环境不能跨机器复制，也不要从 `D:\dev\projects\a` 移动到 `b`；`pyvenv.cfg` 与激活脚本里记录了路径。

**不要复制 .venv 到新电脑**

venv 绑定平台、绝对路径和基础解释器。换电脑后删除旧 `.venv`，用新机器上的 `D:\dev\Python\Python313` 重新创建。真正要迁移的是 `pyproject.toml` / 锁定文件。

DEFAULT

**推荐配置**

项目内 `.venv`；创建时 `--upgrade-deps`；不启用 `--system-site-packages`。

OPTIONAL

**可选参数**

`--copies` 复制解释器；`--prompt` 自定义提示符；`--without-pip` 仅特殊场景。

## 5.4 环境验收与常见误用

- [ ] **路径。**激活后 `sys.prefix` 指向 `D:\dev\projects\demo\.venv`。
- [ ] **pip。** `python -m pip --version` 路径在项目 `.venv` 内。
- [ ] **可重建。**删除 `.venv` 后能用 `py -3.13 -m venv` 重建。
- [ ] **基础解释器。** `sys.base_prefix` 指向 `D:\dev\Python\Python313`。
- [ ] **隔离。**未启用 `--system-site-packages`，全局包不应出现在 `pip list`。
- [ ] **不移动。** `.venv` 不跨目录、不跨机器复制。

**不要为了“省空间”共用 .venv**

两个项目共用环境，短期省几百 MB，长期会让依赖版本互相绑架。真正的空间大头通常不是 venv，而是 pip 缓存、旧解释器和数据集。

PIP / DEPENDENCIES

06

# pip、依赖与可复现

装包时最容易犯的错，是“pip 指向了另一个 Python”。统一使用 `python -m pip`，并把依赖声明与锁定文件放在项目里，才能在新机器上重建。

预计 22 分钟 | pip 25+ | requirements / pyproject

## 6.1 永远用 python -m pip

```
# 推荐：与当前解释器绑定
(.venv) PS> python -m pip install requests

# 不推荐：pip 可能来自另一个环境
(.venv) PS> pip install requests
```

验证 pip 属于哪个环境：

```
(.venv) PS> python -c "import sys; print(sys.executable)"
(.venv) PS> python -m pip --version
pip 25.2 from D:\dev\projects\demo\.venv\Lib\site-packages\pip (python 3.13)
```

如果 pip 路径不在 `D:\dev\projects\demo\.venv` 下，说明环境没激活或 PATH 有另一个 pip。此时继续 `pip install` 很可能装进全局或另一个项目。

## 6.2 requirements 与 pyproject

### requirements.txt：简单项目

```
# requirements.in（直接依赖）
requests>=2.32,<3
python-dotenv>=1.0,<2

# 生成锁定文件
PS> python -m pip install pip-tools
PS> pip-compile requirements.in -o requirements.txt
PS> python -m pip install -r requirements.txt
```

### pyproject.toml：现代项目

```
[project]
name = "demo"
version = "0.1.0"
requires-python = ">=3.13,<3.16"
dependencies = ["requests>=2.32,<3"]

[project.optional-dependencies]
dev = ["pytest>=8", "ruff>=0.9"]

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

```
(.venv) PS> python -m pip install -e ".[dev]"
```

`-e` 是可编辑安装：源码改动立即生效。不要提交 `.venv`；应用项目提交锁定文件，库项目在 `pyproject.toml` 写版本范围。

> **PowerShell 5.1 的 > 会写 UTF-16**
>
> `python -m pip freeze > requirements.lock.txt` 在 Windows PowerShell 5.1 里可能生成 UTF-16 文件，导致其他工具读取异常。改用 `python -m pip freeze | Set-Content -Encoding utf8 requirements.lock.txt`，或使用 PowerShell 7。

## 6.3 镜像、代理与证书

```
# 临时镜像
PS> python -m pip install -i https://pypi.tuna.tsinghua.edu.cn/simple requests

# 写入用户配置
PS> python -m pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
PS> python -m pip config list -v

# 代理 / 企业 CA
PS> python -m pip install --proxy http://127.0.0.1:7890 requests
PS> python -m pip config set global.cert D:\dev\tools\certs\company-ca.pem
```

出现 SSL 错误时，先确认系统时间、代理与企业根证书，不要用 `--trusted-host` 长期绕过校验。私有包应使用公司内部索引，不要让公共镜像有机会提供同名包。

```
# 检查依赖冲突
(.venv) PS> python -m pip check
(.venv) PS> python -m pip list --outdated
```

WORKFLOW / IDE

07

# 项目工作流与 IDE

有了 `D:\dev\projects` 与项目内的 `.venv`，接下来是让终端、VS Code、PyCharm 与 Jupyter 都指向同一个解释器。不要让编辑器用一个版本、运行用另一个版本。

预计 18 分钟 | VS Code / PyCharm | Jupyter

## 7.1 一个项目一个 .venv

```
PS> cd D:\dev\projects\demo
PS> .\.venv\Scripts\Activate.ps1
(.venv) PS> python app.py
(.venv) PS> deactivate
```

如果脚本忘记激活，直接用绝对路径：

```
PS> D:\dev\projects\demo\.venv\Scripts\python.exe app.py
```

项目之间不要共用 `.venv`。两个项目依赖冲突时，拆环境比强行升级更省时间。

## 7.2 目录结构与 .gitignore

```
D:\dev\projects\demo\
├── .venv/# 不提交
├── src/demo/
├── tests/
├── pyproject.toml
├── requirements.txt
├── .env.example
├── .gitignore
└── README.md
```

```
# .gitignore
.venv/
__pycache__/
*.py[cod]
.pytest_cache/
.ruff_cache/
build/
dist/
*.egg-info/
.env
.env.*
!.env.example
```

已经提交了 `.venv`：先加入 `.gitignore`，再 `git rm -r --cached .venv`，提交“停止跟踪”。历史里的大文件不会自动消失，必要时再清理历史。

## 7.3 VS Code、PyCharm、Jupyter

### VS Code

1. 打开 `D:\dev\projects\demo`。
2. `Ctrl+Shift+P` → **Python: Select Interpreter**。
3. 选择 `D:\dev\projects\demo\.venv\Scripts\python.exe`。
4. 新建终端，确认提示符前有 `(.venv)`。

```
# .vscode/settings.json
{
  "python.defaultInterpreterPath": ".venv/Scripts/python.exe",
  "python.terminal.activateEnvironment": true
}
```

如果要同时维护 Windows 与 macOS，设置里不要写死 `D:\dev\...`；用相对路径或把绝对路径留给本机设置。

### PyCharm

Settings → Project → Python Interpreter → Add Interpreter → Existing，选择 `D:\dev\projects\demo\.venv\Scripts\python.exe`。运行配置与终端都应使用这个解释器。

### Jupyter

```
(.venv) PS> python -m pip install ipykernel
(.venv) PS> python -m ipykernel install --user --name demo-3.13 --display-name "Python 3.13 (demo)"
PS> jupyter kernelspec list
```

Notebook 里如果仍找不到包，检查 kernel 的 `sys.executable` 是否指向 `D:\dev\projects\demo\.venv`，而不是只看显示名称。

MULTI-VERSION / TOOLCHAINS

08

# 多版本并用与工具链

同一台机器可以同时保留 3.11、3.13、3.15。关键是让每个项目明确绑定一个版本，并给 Java、Node 等工具留下互不冲突的目录。这一章把 Python 的多版本方法扩展到整个开发盘。

预计 20 分钟 | tox / uv / conda | Java / Node 预留

## 8.1 测试多个版本

库项目用 tox 或 nox 建矩阵，不要手动建五个环境。项目声明支持范围：

```
[project]
requires-python = ">=3.11,<3.16"
```

```
# tox.ini
[tox]
env_list = py311, py313, py315

[testenv]
deps = pytest
commands = pytest -q
```

```
PS> python -m pip install tox
PS> tox
PS> tox -e py313
```

tox 会调用 `python3.11`、`python3.13`、`python3.15`；Windows 上需要这些版本已注册，并且命令能找到。CI 中可以直接用 setup-python 配置同一矩阵。

## 8.2 uv 与 conda

### uv：快速装解释器与锁依赖

```
PS> winget install --id=astral-sh.uv -e
PS> uv python install 3.13 3.15
PS> cd D:\dev\projects\demo
PS> uv python pin 3.13
PS> uv add requests
PS> uv run python app.py
```

uv 默认仍会在项目里创建 `.venv`，但会自动选择/下载解释器并锁定依赖。团队要统一：不要提交 `uv.lock` 后又让其他人用 pip 猜版本。

### conda：数据科学与非 Python 依赖

```
PS> conda create -n py313 python=3.13 numpy pandas
PS> conda activate py313
PS> python -m pip install some-package
PS> conda deactivate
PS> conda env remove -n py313
```

`base` 只做启动器，不装项目依赖。不要在 conda 环境里再建 venv；两层隔离叠加会让 `sys.base_prefix`、PATH 与包来源变得难以解释。

## 8.3 同时配置 Java / Node 的目录

现在把 `D:\dev` 规划好，未来装 JDK、Maven、Node 时就不用再改结构：

```
D:\dev\
├── Python\
│   ├── Python313\
│   └── Python315\
├── Java\
│   ├── jdk-21\          # JAVA_HOME 指向这里
│   ├── jdk-17\
│   └── maven\           # MAVEN_HOME
├── Node\
│   └── node-v22\        # 或由 nvm 管理
├── tools\
└── projects\
```

Java 的配置思路与 Python 类似：

- **JDK**：每个大版本一个目录，例如 `D:\dev\Java\jdk-21`；`JAVA_HOME` 指向当前主版本，PATH 只放 `%JAVA_HOME%\bin`。
- **Maven / Gradle**：放 `D:\dev\Java\maven`、`D:\dev\Java\gradle`；本地仓库可设到 `D:\dev\Java\.m2`，避免占满 C 盘。
- **Node**：可以用 nvm 管理，版本目录也放在 `D:\dev\Node`；不要把全局 npm 包与项目依赖混在一起。
- **工具链**：Git、CMake、数据库客户端放 `D:\dev\tools`，不要放进 Python 或 Java 的目录。

> **目录规划的共同原则**
>
> 工具按“产品 → 版本”分层，项目放 `projects`，缓存与仓库放独立目录。这样任何一个工具升级，都不会影响其他工具或项目。

VERIFY / TROUBLESHOOT

09

# 验收、诊断与排错

装到 `D:\dev\Python\Python313` 只是第一步。验收要确认：py 能看到它、venv 能绑定它、pip 装在项目里、IDE 与终端用的是同一个解释器。

预计 18 分钟 | 12 项验收 | 15 个报错

## 9.1 十二项验收

| # | 检查 | 命令 | 通过标准 |
|---|---|---|---|
| 1 | py 可见 | `py -0p` | 显示 `D:\dev\Python\Python313` |
| 2 | 版本正确 | `py -3.13 --version` | 返回 3.13.x |
| 3 | 解释器路径 | `py -3.13 -c "import sys; print(sys.executable)"` | 指向 D:\dev\Python\Python313 |
| 4 | pip 归属 | `py -3.13 -m pip --version` | 路径在 Python313 下 |
| 5 | venv 创建 | `py -3.13 -m venv .venv` | 项目下生成 .venv |
| 6 | 激活 | `.\.venv\Scripts\Activate.ps1` | 提示符出现 (.venv) |
| 7 | 环境绑定 | `python -c "import sys; print(sys.prefix); print(sys.base_prefix)"` | prefix 是项目，base 是 Python313 |
| 8 | 包可安装 | `python -m pip install requests` | 成功且无权限错误 |
| 9 | 包可导入 | `python -c "import requests; print(requests.__version__)"` | 输出版本号 |
| 10 | 依赖一致 | `python -m pip check` | No broken requirements |
| 11 | 脚本可运行 | `python app.py` | 项目入口正常 |
| 12 | 退出复原 | `deactivate` | 回到系统入口 |

**表 9-1** 在 `D:\dev\projects\demo` 的临时项目中执行第 5–12 项。

## 9.2 诊断命令

```
# 入口与路径
PS> py -0p
PS> where.exe python
PS> py -3.13 -c "import sys; print(sys.executable); print(sys.path)"

# 环境与 pip
(.venv) PS> python -c "import sys; print(sys.prefix); print(sys.base_prefix)"
(.venv) PS> python -m pip --version
(.venv) PS> python -m pip check

# 配置来源
PS> python -m pip config list -v
```

一条命令生成环境快照：

```
Python: 3.13.7
sys.executable: D:\dev\projects\demo\.venv\Scripts\python.exe
sys.prefix: D:\dev\projects\demo\.venv
sys.base_prefix: D:\dev\Python\Python313
pip: D:\dev\projects\demo\.venv\Lib\site-packages\pip
PY_PYTHON: 3.13
pip check: No broken requirements found.
```

## 9.3 十五个高频报错

01 **“python 不是内部或外部命令”**

多版本机器不要依赖 `python`，用 `py -3.13`。如果必须用，只给主版本加 PATH，并重开终端。

02 **输入 python 弹出 Microsoft Store**

关闭应用执行别名中的 `python.exe` / `python3.exe`。

03 **py -3.13 找不到**

安装时没勾 py launcher，或装到了别的账户。重装到 `D:\dev\Python\Python313`，勾选 launcher。

04 **装了 3.15，py -0p 里却只有旧版本**

确认安装路径与 launcher；`py -0p` 显示的是已注册版本，便携版不会自动出现。

05 **Activate.ps1 被禁止运行**

`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`，重开 PowerShell；不要长期用 Bypass。

06 **venv 创建失败 / ensurepip 不可用**

Windows 重装 Python 并保留 pip；Linux 装 `python3-venv` 或 `python3.13-venv`。

07 **激活后 pip 还是装到全局**

用 `python -m pip --version` 看路径；不在 `D:\dev\projects\demo\.venv` 就是入口不对。

08 **安装成功但运行报 ModuleNotFoundError**

安装与运行不是同一个解释器。分别打印 `sys.executable`，检查 IDE 与 Jupyter kernel。

09 **把 .venv 移到别的目录后不能用**

venv 绑定路径，不能移动。删除后用 `py -3.13 -m venv .venv` 重建。

10 **Could not find a version / 没有 wheel**

确认包名、Python 版本与平台；新版本 Python 可能还没有对应的二进制包。

11 **SSL certificate verify failed**

公司根证书或代理问题。配置 `pip config set global.cert ...`，不要关闭校验。

12 **两个依赖版本冲突**

用 `pip check` 定位；拆环境或选兼容版本，不要 `--force-reinstall`。

13 **D 盘安装时提示权限不足**

给当前账户 `D:\dev` 修改权限，或用管理员建目录后再用普通用户安装。

14 **pyenv 切了版本但 IDE 还是旧的**

IDE 的解释器是单独设置，重新 Select Interpreter，选 pyenv 版本的实际路径。

15 **conda activate 不生效**

运行 `conda init` 并重开 shell；不要让 conda 与 pyenv 的 shim 抢同一个 python。

MAINTENANCE / MIGRATION

10

# 维护、迁移与清理

目录固定后，维护会简单很多：版本在 `D:\dev\Python`，项目在 `D:\dev\projects`，缓存与工具在各自目录。升级、换电脑、清理都可以按目录处理。

预计 12 分钟 | 长期适用 | 迁移清单

## 10.1 版本升级与回滚

| 类型 | 操作 | 注意 |
| --- | --- | --- |
| 补丁升级 | 覆盖安装到原目录 | 3.13.7 → 3.13.8，保留配置与包。 |
| 次版本升级 | 新装到 `Python315` | 不覆盖 3.13；项目逐个重建环境。 |
| 回滚 | 卸载新版本，保留旧目录 | 旧 venv 不跨版本，重建即可。 |

**表 10-1** 升级策略。生产环境先在副本上跑测试。

升级前记录 `py -0p` 与项目 `requirements`；升级后用第 9 章验收。不要把旧版本目录删掉再装新版本，否则回滚会很麻烦。

## 10.2 换电脑迁移清单

- [ ] **目录结构。**新机器创建 `D:\dev` 与各子目录。
- [ ] **解释器。**按第 3 章重新安装，不复制。
- [ ] **密钥。**通过密码管理器迁移，不放进项目。
- [ ] **项目。**从 Git 克隆到 `D:\dev\projects`。
- [ ] **依赖。**用 `pyproject.toml` / 锁文件重建。
- [ ] **验收。**在新机跑 12 项检查。

不要迁移 `.venv`、Python 安装目录或 `__pycache__`。这些都是本机生成物，跨机器复制只会带来路径问题。

## 10.3 清理与磁盘管理

```
# pip 缓存
PS> python -m pip cache dir
PS> python -m pip cache purge

# 查看 D:\dev 占用（PowerShell）
PS> Get-ChildItem D:\dev -Directory | ForEach-Object {
  "{0}  {1:N1} GB" -f $_.Name, ((Get-ChildItem $_.FullName -Recurse -File | Measure-Object Length -
Sum).Sum / 1GB)
}
```

删除不再使用的项目时，连同其 `.venv` 一起删；删除解释器时用安装器卸载。保留 `D:\dev\projects` 的目录结构，未来 Java、Node 也可以复用。

## 10.4 D:\dev 目录维护清单

| 目录 | 清理什么 | 不要做什么 |
| --- | --- | --- |
| `D:\dev\Python\Python3xx` | 不再使用的整版本 | 直接删目录；用安装器卸载。 |
| `D:\dev\projects\项目\.venv` | 废弃环境、重建前的旧环境 | 跨机器复制；在其中装全局工具。 |
| `D:\dev\projects\项目` | 已归档项目的整个目录 | 删除未推送的提交与未备份数据。 |
| `D:\dev\Java` | 旧 JDK、旧 Maven 缓存 | 删除正在被 `JAVA_HOME` 使用的版本。 |
| `D:\dev\tools` | 旧安装包、临时解压目录 | 把 Python 包或项目代码放进来。 |

**表 10-2** 按目录清理。每次只删一个明确不再使用的对象，并先确认没有环境变量指向它。

**每季度做一次目录盘点**

查看 `D:\dev` 各目录占用，删除废弃的 venv 与旧安装包；保留至少一个可回滚的 Python 版本。目录干净，升级时更不容易误删。

APPENDIX A / CHEAT SHEET

A

# 命令速查

按“目录 → 解释器 → venv → 依赖 → 诊断”分组。路径按本指南约定写成 `D:\dev`，盘符可按实际机器替换。

可打印 跨平台 D:\dev 约定

| 命令 | 作用 |
| --- | --- |
| `New-Item -ItemType Directory D:\dev\Python` | 创建 Python 安装根目录。 |
| `py -0p` | 查看 Windows 已注册解释器与路径。 |
| `py -3.13 --version` | 调用指定版本。 |
| `py -3.13 -m venv D:\dev\projects\demo\.venv` | 在项目内创建环境。 |
| `.\.venv\Scripts\Activate.ps1` | PowerShell 激活。 |
| `source .venv/bin/activate` | macOS / Linux 激活。 |
| `python -m pip install -r requirements.txt` | 安装项目依赖。 |
| `python -m pip freeze \| Set-Content -Encoding utf8 requirements.lock.txt` | PowerShell 安全导出锁定文件。 |
| `python -m pip check` | 检查依赖冲突。 |
| `python -c "import sys; print(sys.executable)"` | 查看当前解释器。 |
| `uv python install 3.13` | uv 管理解释器。 |
| `conda create -n py313 python=3.13` | conda 创建环境。 |

**表 A-1** 最常用命令。所有 pip 安装都应在目标环境中执行。

APPENDIX B / SCRIPTS

B

# 初始化脚本

脚本按 `D:\dev\projects` 约定创建环境，不写死用户名。运行前先读一遍，确认版本与路径。

PowerShell Bash D:\dev

## B.1 Windows PowerShell

```powershell
# init-python.ps1
param(
    [string]$Version = "3.13",
    [string]$Project = "demo"
)
$ErrorActionPreference = "Stop"

$root = "D:\dev\projects\$Project"
New-Item -ItemType Directory -Force $root | Out-Null
Set-Location $root

if (Test-Path ".venv") {
    throw ".venv already exists; remove it explicitly before recreating."
}

py "-$Version" -m venv --upgrade-deps .venv
$python = Join-Path $root ".venv\Scripts\python.exe"
& $python -m pip install --upgrade pip
if (Test-Path "requirements.txt") {
    & $python -m pip install -r requirements.txt
}
& $python -c "import sys; print(sys.executable); print(sys.version)"
Write-Host "Done: $root\.venv"
```

运行 `.\init-python.ps1 -Version 3.13 -Project demo`。脚本不会删除已有环境，避免误删依赖。

## B.2 macOS / Linux Bash

```bash
#!/usr/bin/env bash
set -euo pipefail
VERSION="${1:-3.13}"
PROJECT="${2:-demo}"
ROOT="$HOME/dev/projects/$PROJECT"

mkdir -p "$ROOT"
cd "$ROOT"
if [ -e .venv ]; then
  echo ".venv already exists; remove it explicitly first." >&2
  exit 1
fi

"python$VERSION" -m venv --upgrade-deps .venv
".venv/bin/python" -m pip install --upgrade pip
[ -f requirements.txt ] && ".venv/bin/python" -m pip install -r requirements.txt
".venv/bin/python" -c 'import sys; print(sys.executable); print(sys.version)'
```

运行 `./init-python.sh 3.13 demo`。macOS 的 `~/dev/projects` 与 Windows 的 `D:\dev\projects` 结构对应。

APPENDIX C / REFERENCE

# 版本、资源与术语

最后放版本支持、官方链接与术语。路径规范按本版 `D:\dev` 约定执行。

2026-10 | Python 3.15.2 | 术语 12 条

## C.1 版本支持

| 版本 | 状态 | 安全支持至 | 建议 |
| --- | --- | --- | --- |
| **3.15** | bugfix | 2031-10 | 尝鲜；确认依赖有 wheel。 |
| **3.14** | bugfix | 2030-10 | 新项目候选。 |
| **3.13** | security | 2029-10 | 主力基线。 |
| **3.12** | security | 2028-10 | 旧项目兼容。 |
| **3.11** | security | 2027-10 | 维护旧项目。 |
| **3.10** | EOL | 2026-10-01 | 不要开新项目。 |

**表 C-1** 截至 2026-10 的支持周期；以 python.org 与 PEP 为准。

## C.2 官方资源

**Python 下载**
https://www.python.org/downloads/
Windows/macOS 安装器与版本列表。

**venv 文档**
https://docs.python.org/3/library/venv.html
创建参数与 pyvenv.cfg。

**pip 文档**
https://pip.pypa.io/en/stable/
安装、配置、缓存与依赖解析。

**uv 文档**
https://docs.astral.sh/uv/
解释器、项目与锁文件。

**pyenv**
https://github.com/pyenv/pyenv
多版本安装与 shell 初始化。

**conda**
https://docs.conda.io/
数据科学环境与非 Python 依赖。

## C.3 术语

| 术语 | 解释 |
| --- | --- |
| **解释器** | 实际执行 Python 代码的程序，例如 `D:\dev\Python\Python313\python.exe`。 |
| **venv** | 标准库虚拟环境；隔离包与入口，不复制标准库。 |
| **pyvenv.cfg** | 记录 venv 的基础解释器与版本。 |
| **py 启动器** | Windows 按版本号选择解释器的工具。 |
| **site-packages** | 第三方包目录；venv 内有独立副本。 |
| **PATH** | 系统查找命令的目录顺序。 |
| **wheel** | 预编译二进制包；缺失时可能源码编译。 |
| **PEP 668** | 系统 Python 的 externally managed 保护。 |
| **lock file** | 锁定完整依赖解析结果。 |
| **editable install** | `pip install -e`，源码改动立即生效。 |
| **Jupyter kernel** | Notebook 实际使用的解释器会话。 |
| **EOL** | 停止安全支持的版本终点。 |

**表 C-2** 阅读文档与排查时的高频术语。

COLOPHON

版本：修订版 v1.1 · 更新：2026 年 10 月 · 基线：Python 3.15.2 · 路径：`D:\dev\Python`

ONE DEV ROOT, MANY TOOLCHAINS

# 版本各归其位，工具各占一层。

`D:\dev\Python` 放解释器，`D:\dev\projects` 放项目，`D:\dev\Java` 与 `D:\dev\Node` 留给下一套工具链。目录清楚，环境就不会互相打架。

01 **解释器：D:\dev\Python\Python3xx**

02 **项目：D:\dev\projects\name**

03 **工具链：Java / Node / tools 各自分层**